GrassKit Rover

Two rovers that mow any lawn on their own: where the build stands and what comes next

AT A GLANCE

Where we are. One rover drives. A four-wheel mecanum base runs our own firmware on an STM32 Nucleo board and has been driven on the bench from a browser over a USB cable.

What is not right yet. Several drive commands move the rover the wrong way, and Bluetooth does not carry commands yet.

What comes next. Fix the wheel directions, then finish Bluetooth so the rover can be driven from a phone with no cable. Sensing, SLAM, the cutting head and the second rover all come after that.

1 The goal

Two small rovers that can be put down on any lawn and cut it on their own: no boundary wire, no map prepared in advance and nobody steering. The rovers have to find the shape of the lawn, agree how to share it, cover every part of it, keep clear of whatever is in the way, and cut as they go.

The work is staged so that each stage ends with something that runs. Stage 1 is nearly complete; stage 2 is the immediate next step.

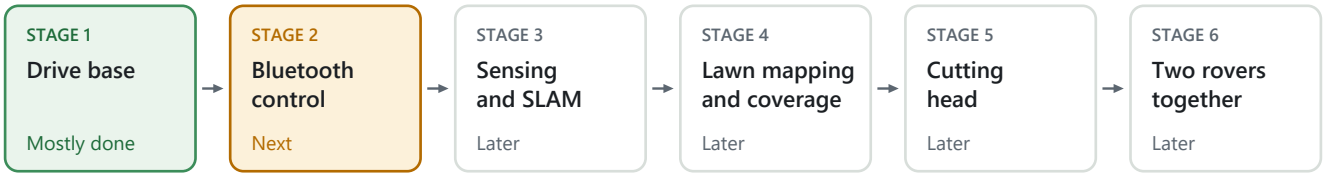


Figure 1. The six stages. Stage 1 still has open issues, listed in section 4.

2 Where the project stands

One rover exists and drives under cable control. Figure 2 shows it as built; the table that follows lists every area of the project and how far it has got.

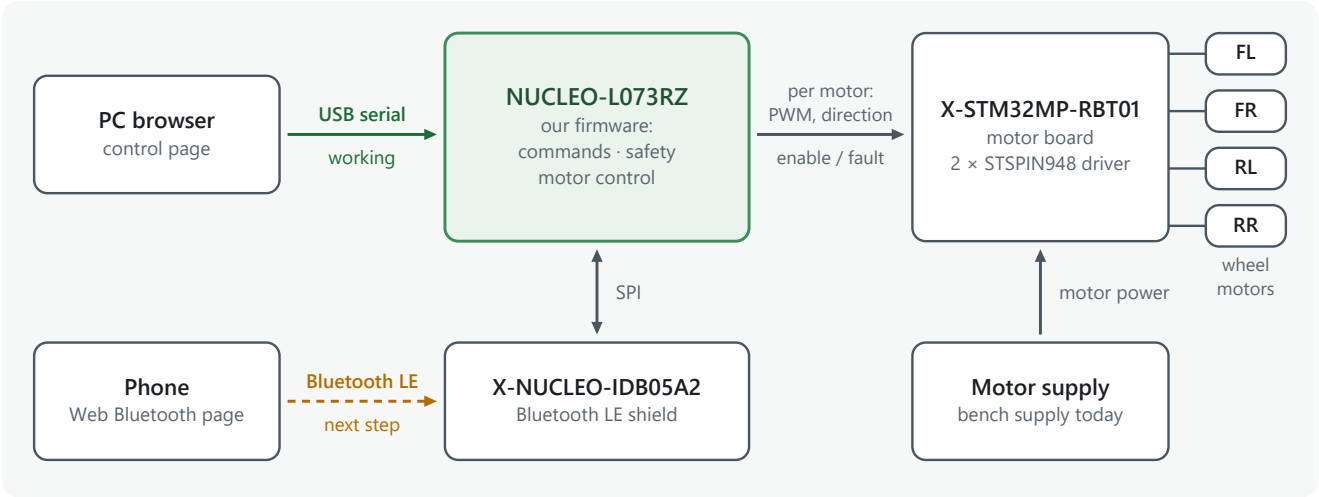


Figure 2. The rover today. The USB path is what all testing has used; the Bluetooth path is the next step.

Area	Status	Notes
Hardware chosen and wired	Done	Nucleo-L073RZ controller, X-STM32MP-RBT01 motor board, Bluetooth shield, four-motor mecanum chassis
Pin map checked against ST manuals	Done	Rebuilt from ST's board manual and driver datasheet after the first table proved wrong
Motor firmware	Done	PWM, direction, enable, ramping and mecanum mixing; run on the bench
Safety layer	Done	Safe start-up, command timeout, emergency stop, driver-fault detection. No hardware watchdog yet
Command protocol and USB control page	Done	Used for all bench testing so far
Vendor documents and repository	Done	43 PDFs indexed in the documentation folder; code on GitHub
Wheel direction mapping	Open issue	Rear-left wheel runs backwards; sideways and turn commands move the wrong way
Bluetooth control	In progress	The shield answers over SPI. It is not confirmed advertising and has no data service
Battery power	Not started	Bench supply so far; batteries still to be chosen
Odometry and sensing	Not started	No wheel encoders; the motor board's sensors are not read yet
SLAM, depth maps, lawn mapping	Not started	Needs a larger computer than the Nucleo
Cutting head	Not started	Blade or trimmer line still to be decided
Machine learning	Not started	Grass, lawn-edge and obstacle recognition
Second rover	Not started	A copy of the first, plus coordination between the two

3 What has been done

3.1 The rover as built

The rover is assembled from ST evaluation boards on a four-wheel chassis, connected as in Figure 2. Nothing is custom-made yet, which keeps every part documented and replaceable.

Part	What it is	What it does here
NUCLEO-L073RZ	STM32L073RZ microcontroller board: Cortex-M0+ at 32 MHz, 192 KB flash, 20 KB RAM	Runs all of the firmware
X-STM32MP-RBT01	ST robotics board with two STSPIN948 dual full-bridge motor drivers. It also carries an inertial sensor, a magnetometer, a pressure sensor and a time-of-flight ranging sensor	Drives the four wheel motors. Its sensors are not used yet
X-NUCLEO-IDB05A2	Bluetooth Low Energy 4.2 shield built on the BlueNRG-M0 module, connected over SPI	Wireless link to a phone. Not working yet
Chassis	Four DC gear motors with mecanum wheels	Lets the rover move forwards, sideways and rotate
Power	USB for the Nucleo, a bench supply for the motors	Batteries are still to be chosen

3.2 Getting the wiring right

The RBT01 was designed to plug onto an STM32MP discovery kit, not a Nucleo, so every signal is jumper-wired from the Nucleo to the board's 40-pin header. The project's first pin table had been written without ST's manual and did not match the board. It was rebuilt from the board's user manual (UM3482, Table 11 and the schematic sheets) and the STSPIN948 datasheet, and the firmware, the CubeMX configuration and every wiring document were changed to match.

Three findings from that work shape the firmware:

- **Each motor needs three lines:** a PWM line for speed, a phase line for direction, and a combined enable and fault line.
- **The drivers are on by default.** The board pulls every enable line high, so a motor can be driven as soon as its supply is present. The firmware therefore holds all four lines low at power-up and whenever the rover is idle.
- **The enable line is also the fault output.** The driver pulls it low on under-voltage, over-current or over-temperature, so the firmware drives it open-drain and reads it back.

Motor	Speed (PWM)	Direction	Enable / fault	Motor terminals
1 Front-left	`PB4 → pin 32`	`PC0 → pin 23`	`PC4 → pin 13`	`CN4: 2A + 1A`
2 Front-right	`PB5 → pin 33`	`PC1 → pin 24`	`PC5 → pin 15`	`CN4: 1B + 2B`
3 Rear-left	`PB0 → pin 31`	`PC2 → pin 19`	`PC6 → pin 16`	`CN5: 2A + 1A`
4 Rear-right	`PB1 → pin 36`	`PC3 → pin 21`	`PC8 → pin 18`	`CN5: 1B + 2B`

Table: Nucleo pin to RBT01 header pin for each motor. The board's mode jumpers stay at their factory positions.

3.3 Firmware

The firmware is about 2,400 lines of application C on ST's HAL library, built with the GCC toolchain that ships with STM32CubeIDE and flashed through the Nucleo's on-board ST-LINK. It occupies about 79 KB of the 192 KB of flash and under 2 KB of the 20 KB of RAM for static data. Everything runs from one loop that executes once per millisecond.

Module	Responsibility
`app.c`	The one-millisecond loop: read commands, run the safety checks, update the motor ramps, send telemetry, and blink the status LED at a rate that shows the state
`motor_control.c`	PWM, direction and enable for four motors; mecanum mixing; ramping; driver-fault detection
`safety.c`	Command timeout, emergency-stop button, latched emergency stop
`command_parser.c`	The text command protocol
`bluetooth.c`	Serial receive from USB and a spare UART into one buffer; bring-up of the Bluetooth module
`engine_model.c`	A virtual engine (RPM, gears, turbo) left from the project's start as a race-car demo. It feeds telemetry only
`debug.c`	Boot banner and the periodic telemetry line

The project began as a race-car demonstrator with synthesised engine sound. The audio engine has been removed from the firmware and documents. The project folder name, the virtual engine model and three commands (GEAR, TURBO, REV) are what remains of that.

3.4 Motor control

- **PWM.** One timer produces four 20 kHz PWM signals, above the audible range.

- **Speed and direction.** Each wheel takes a signed speed from −100 % to +100 %. Below 4 % the output is held at zero.
- **Ramping.** A wheel's speed moves towards its target by at most 2 % per millisecond, so a standing start to full speed takes 50 ms. This avoids current spikes.
- **Mixing.** Forward, sideways and rotation demands are combined into four wheel speeds with the standard mecanum equations, and all four are scaled down together if any would exceed 100 %. A second mode drives the left and right sides at different speeds, without sideways motion.
- **Stopping.** STOP ramps down and then switches the drivers off. BRAKE zeroes the PWM with the drivers on, which brakes the motors. Emergency stop switches the drivers off at once.
- **Single-wheel test.** One command drives one wheel alone, which is how wiring and direction are checked.

3.5 Safety

- **Safe start-up.** The motor outputs are set up before anything else at boot, and the drivers stay off until a drive command arrives.
- **Off when idle.** Once the wheels have ramped to zero the drivers are switched off again.
- **Command timeout.** If the rover is moving and no valid command arrives for 1.5 seconds, it stops. Unrecognised commands do not count.
- **Emergency stop.** The blue button on the Nucleo or the ESTOP command switches the drivers off and zeroes the PWM. The stop is latched until RESET is sent.
- **Driver faults.** After a 10 ms settling time, three low readings in a row on a motor's fault line switch all drivers off and report which motor tripped.

One gap remains: there is no independent hardware watchdog, so a firmware hang would leave the outputs in their last state.

3.6 Commands and telemetry

Commands are plain text, one per line, at 115200 baud, and are not case-sensitive. Each gets a reply that starts with OK, ERR, WARN or ALARM. The same parser serves the USB serial port and the spare UART, and is where Bluetooth will deliver its data.

Command	Effect
`START, STOP, BRAKE`	Enable the drivers; ramp to a stop; brake
`ESTOP, RESET`	Latched emergency stop; clear it (this also clears a driver fault)
`DRIVE:f,r,s`	Forward, rotation and sideways demand in one line, each from −100 to 100
`THROTTLE:n, STEER:n, STRAFE:n`	Set one of the three demands at a time
`MODE:MECANUM, MODE:RACECAR`	Choose the mixing
`MOTOR:n,speed`	Drive one wheel alone (n is 1 to 4)
`STATUS`	One-line report: state and the four wheel speeds
`BLE_TEST, BLE_ADV`	Bluetooth diagnostics: run the bring-up sequence and report each step; restart advertising
`GEAR, TURBO, REV`	Race-car leftovers. They change the virtual engine only, not the wheels

While the rover is running or a controller is connected, a telemetry line with the four wheel speeds is sent every 200 ms.

3.7 Control tools

Tool	What it is
`web_serial_server.py`	A small Python server that opens the Nucleo's USB serial port and serves a control page in the browser: a drive pad, a test button for each wheel and live telemetry. It keeps sending commands so the command timeout does not trip. All bench testing so far used this page
`racecar_controller.py`	A desktop Python window with sliders, a sideways pad and a log
`web_bluetooth_controller.html`	A phone page that uses Web Bluetooth and expects a Nordic UART Service on the rover. It is waiting for the firmware side
`scan_ble.py`	Scans for the rover's Bluetooth advertisement

3.8 What has run on the bench

- The firmware builds, flashes and boots, and reports its state over USB.
- All four wheels have been driven from the browser control page.
- **A dead front-right wheel was traced to the firmware, not the wiring.** On this microcontroller the pin used for motor 2's PWM is connected to a different timer unless a remap setting is made. The firmware never made it, so motor 2 received no PWM. With motor 2 commanded to 50 %, the pin read low in 12 of 12 samples before the fix and high in 6 of 12 after it.
- The highest voltage and current recorded at the wheels were 6.5 V and 0.45 A.
- Not every command moves the rover the right way yet. That is the first item in section 4.

3.9 Documents and repository

Forty-three vendor PDFs are stored in the documentation folder in nine numbered sub-folders: board manuals, schematics, datasheets, tool guides, ST's material on using the RBT01 from Linux, and one SLAM reference. An index gives each document's number and revision and says what to use it for. The code and documents are in a Git repository on GitHub (nottysukku/GrassKit_Rover).

4 Open issues in what exists

These need closing before the next stage builds on top of them.

4.1 Wheel directions

Driving from the control page gave these results:

Command	What the rover did
Forward	Rear-left wheel runs backwards
Sideways left	Moves right
Sideways right	Moves forwards
Turn left	Moves forwards
Turn right	Moves sideways to the right

Working the mixing equations through with the rear-left motor reversed gives motion consistent with four of the five results (forward, sideways-right, turn-left and turn-right), so that one motor is the likely cause of most of this. It can be corrected by swapping its two leads or by inverting it in firmware. Sideways-left does not fit the same explanation

and should be tested again after the correction; if it is still wrong, the roller pattern of the wheels is the next thing to check.

4.2 Bluetooth does not carry commands

The shield is fitted and its SPI link responds: reset works, the module raises its interrupt line when it boots, and it accepts command headers. The firmware resets the module, initialises it and asks it to advertise under the name RACECAR-L073. Two things are missing. Advertising has not been confirmed from a phone or a scanner, and the firmware defines no data service, so a phone that did connect would have nowhere to send commands.

4.3 Power

The motors have only run from a bench supply. The plan is a 9 V battery for the Nucleo and a separate battery pack for the motor board, sized from the 6.5 V and 0.45 A measurement with margin. The motor pack has not been chosen.

4.4 Smaller items

- The four current-limit trimmers on the motor board should be set to suit the motors and the setting recorded.
- Add a hardware watchdog so that a firmware hang switches the drivers off.
- Remove the race-car leftovers: the virtual engine model, the GEAR, TURBO and REV commands, and the project folder name.

5 What comes next

These are proposals, in the order they are likely to be needed. Each is kept short on purpose; none has been started.

5.1 Bluetooth control (the next step)

- Get the module advertising, and see it in the scanner script and on a phone.
- Add a UART-style data service. The phone page already expects the Nordic UART Service: one characteristic for commands in, one for telemetry out.
- Handle the module's events properly (connected, disconnected, data written) and pass written data to the existing command parser. Stop the rover when the phone disconnects.
- Result: the rover is driven from a phone with no USB cable.

5.2 Battery power and a first drive on grass

- Choose the motor battery, set the current limits and run the rover untethered.
- Try real grass early. Mecanum wheels are made for hard, flat floors, so grip and sideways motion on a lawn need to be measured. If they are poor, treaded wheels with left-right steering are the fallback, and the firmware already has that mode.

5.3 Odometry: knowing how far the rover has moved

- The motors run open-loop today: the firmware sets a duty cycle and never learns how fast a wheel turned. Add wheel encoders, or motors with encoders built in, and closed-loop speed control.
- Read the inertial sensor and magnetometer already on the RBT01 for heading.
- Wheel distance plus heading is an input that every SLAM method needs.

5.4 A larger computer on the rover

- The Nucleo's microcontroller has 20 KB of RAM. It cannot run SLAM, depth processing or neural networks.
- The RBT01 was designed to sit on an STM32MP157F-DK2 or STM32MP257F-DK running Linux, and ST's X-LINUX-RBT1 package drives its motors and sensors from there. Its header also matches the Raspberry Pi layout.

- Decision to make: move motor control onto the Linux board, or keep the Nucleo as the real-time motor controller and have the Linux board send it the same text commands. Figure 3 shows the second option.

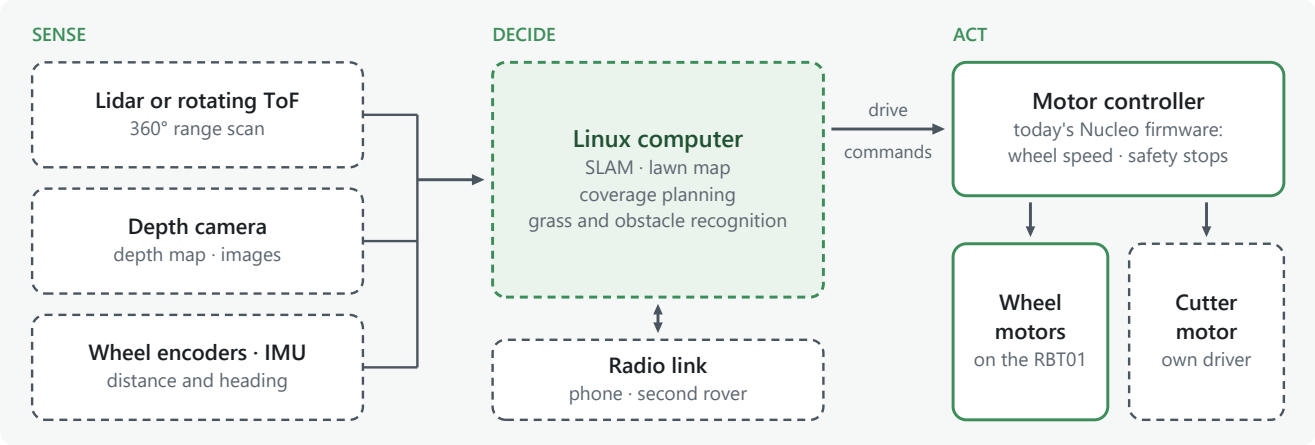


Figure 3. A proposed layout for each rover. Solid outlines exist today; dashed outlines are still to be added.

5.5 SLAM and depth maps

- **Range sensor.** SLAM needs a scan of distances all around the rover. A ready-made spinning 2D lidar gives that directly. The alternative is to build the rotator: a small motor turning a time-of-flight sensor, with an angle sensor and a slip ring for the wiring. Buying is simpler and more accurate; building is cheaper and reuses ST parts.
- **Depth.** A stereo or time-of-flight depth camera gives a depth map of what is ahead. The 8 × 8 zone time-of-flight sensor on the RBT01 can cover the area just in front of the rover.
- **Software.** ROS 2 with slam_toolbox for mapping and Nav2 for navigation is the usual starting point.
- **Lawn caveat.** Open grass gives a lidar little to lock onto, and sunlight shortens time-of-flight range. Expect to combine the scan with odometry and the inertial sensor, and possibly to add satellite positioning (RTK) or fixed markers.

5.6 Mapping an unknown lawn and covering it

- **Find the boundary.** Either drive the edge once by remote control and record it, or detect the edge of the grass with the camera.
- **Store the map** as a grid of cells marked lawn, obstacle or keep-out (flower beds, ponds).
- **Plan coverage.** Split the lawn into simple regions and sweep each in back-and-forth stripes with a small overlap, recording what has been cut.
- **Recover.** Find the rover's position again after a restart, and cope with slopes and wheel slip.

5.7 Cutting head

- **Blade or line.** Small pivoting blades on a spinning disc cut cleanly and need little power, but must be guarded. Trimmer line tolerates stones and edges and is gentler on contact, but uses more power, wears and is louder.
- **Its own motor and driver.** All four outputs of the RBT01 are taken by the wheels, so the cutter needs a separate driver. Motor current can be used to detect a stalled or blocked cutter.
- **Placement.** Under the chassis on the centre line, between the axles, with adjustable height. The cutting width sets the stripe spacing for the coverage planner.
- **Safety.** A guard, a lift and tilt sensor that stops the cutter, and an emergency stop that removes cutter power as well as wheel power.

5.8 Machine learning

- **Grass or not grass** in the camera image, to follow the lawn edge and stay on the lawn.
- **Cut or uncut grass,** to check coverage.

- **Obstacles:** people, pets, toys, hoses. Stop for some, steer round others.
- **What it needs:** labelled images collected by the rover's own camera on several lawns, and models small enough to run on the on-board computer.

5.9 Two rovers

- A shared map in one coordinate frame, a way to divide the lawn between the rovers, and a rule for keeping clear of each other.
- A radio link between the rovers. Bluetooth stays as the link to the phone.
- Build one complete rover first. The second is a copy plus the coordination layer.

5.10 Ready for the field

- Weather sealing, battery life, and a charging dock the rover can return to.
- The safety standard for robotic lawnmowers, IEC 60335-2-107, as the reference for guards, sensors and stopping behaviour.

6 Suggested order of work

Step	Work	Done when
1	Fix wheel directions	Every move on the drive pad goes the commanded way
2	Bluetooth control	The rover is driven from a phone with no cable
3	Battery power	The rover runs untethered, and has been tried on grass
4	Encoders and heading	The rover reports distance and heading and can drive a measured square
5	Linux computer on board	ROS 2 runs on the rover and its drive commands reach the wheels
6	Range sensor and SLAM	The rover maps a room, then a lawn
7	Boundary and coverage planning	The rover covers a whole lawn with the cutter off
8	Cutting head	The rover cuts while it covers
9	Recognition models	The rover stops for obstacles and follows the grass edge
10	Second rover	Two rovers share one lawn